

Figures and Tables

Baijia Xu

04/14

```
library(dplyr)
library(ggplot2)
library(kableExtra)
```

Simulation Study

Local Demo

```
source(here::here("R", "em_functions.R"))
source(here::here("R", "simulate_data.R"))

# simulate data
dat = sim_lcr_data(n = 100, p = 6, K = 2, seed = 42)

cat(sprintf("n = %d, p = %d, q = %d, K = %d\n", nrow(dat$s_mat), dat$p, dat$q, dat$K))

## n = 100, p = 6, q = 21, K = 2

cat(sprintf("True state frequencies: %s\n", paste(table(dat$true_z), collapse = ", ")))

## True state frequencies: 44, 56

# Best-of-20 fit (K-means init): used for all subsequent inference
fit = em_lcr(dat$s_mat, dat$r_mat, K = 2, n_init = 20, tol = 1e-4, max_iter = 500)

cat(sprintf("Converged: %s in %d iterations\n", fit$converged, fit$n_iter))

## Converged: TRUE in 20 iterations

cat(sprintf("Final log-likelihood: %.2f\n", fit$loglik[fit$n_iter]))

## Final log-likelihood: -482.13
```

Log-Likelihood Convergence

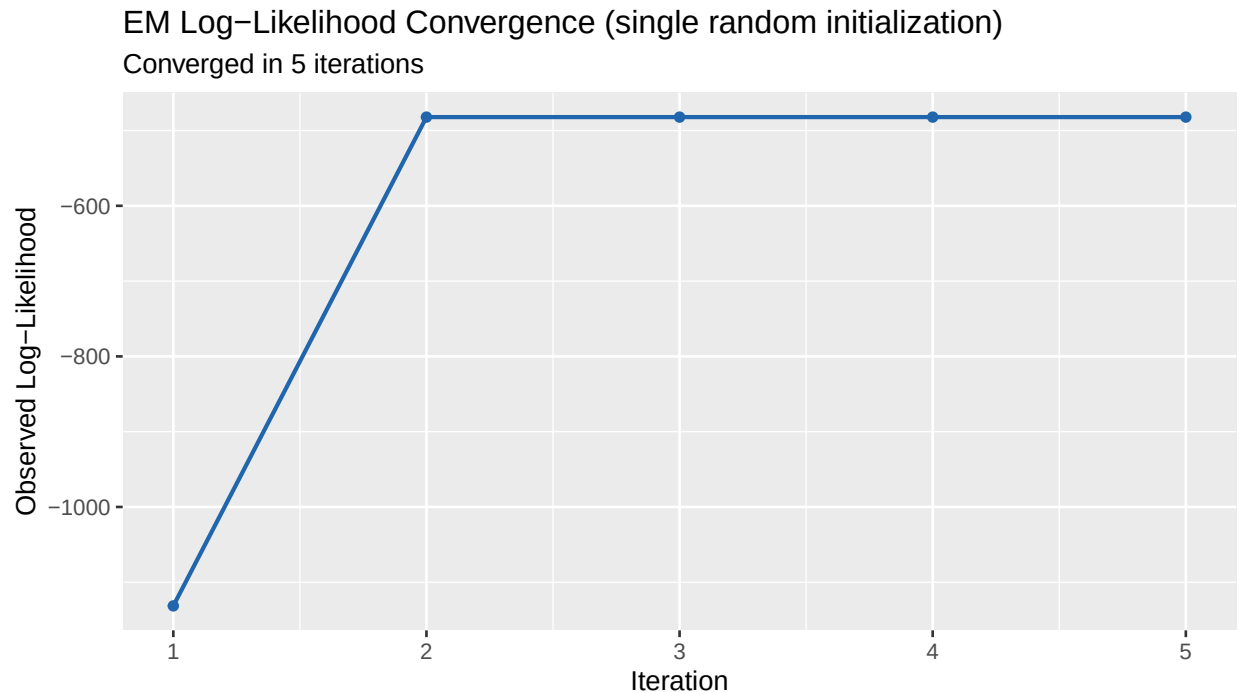
To illustrate EM convergence dynamics, we run a single instance from a random initialization (not the best-of-20 K-means fit above). This shows the full path from a cold start to convergence.

```

# Single run from random initialization - shows convergence path
fit_conv = em_lcr(dat$s_mat, dat$r_mat, K = 2, n_init = 1,
                 init_method = "random", tol = 1e-4, max_iter = 500, min_iter = 5)

tibble(
  iteration = 1:fit_conv$n_iter,
  loglik = fit_conv$loglik
) %>%
  ggplot(aes(x = iteration, y = loglik)) +
  geom_line(linewidth = 0.8, color = "#2166AC") +
  geom_point(size = 1.5, color = "#2166AC") +
  labs(
    title = "EM Log-Likelihood Convergence (single random initialization)",
    subtitle = sprintf("Converged in %d iterations", fit_conv$n_iter),
    x = "Iteration",
    y = "Observed Log-Likelihood"
  )

```



Parameter Recovery

```

# align estimated parameters to true parameters
aligned = align_labels(fit$params, dat$true_params, K = 2)

# Each state has  $q = p(p+1)/2 = 21$  beta0 and 21 beta1 values (one per edge).
# The diagonal simplification means edge j's task connectivity depends only on
# edge j's rest connectivity -- no cross-edge effects.
# We show the first 10 edges here; the scatter plot below shows all q edges.
n_show = min(10, dat$q)

```

```

param_table = bind_rows(
  tibble(
    state      = "State 1",
    edge       = 1:n_show,
    beta0_true = round(dat$true_params$beta0[1, 1:n_show], 4),
    beta0_est  = round(aligned$beta0[1, 1:n_show], 4),
    beta1_true = round(dat$true_params$beta1[1, 1:n_show], 4),
    beta1_est  = round(aligned$beta1[1, 1:n_show], 4)
  ),
  tibble(
    state      = "State 2",
    edge       = 1:n_show,
    beta0_true = round(dat$true_params$beta0[2, 1:n_show], 4),
    beta0_est  = round(aligned$beta0[2, 1:n_show], 4),
    beta1_true = round(dat$true_params$beta1[2, 1:n_show], 4),
    beta1_est  = round(aligned$beta1[2, 1:n_show], 4)
  )
)

param_table %>%
  kable(
    caption = sprintf(
      "Estimated vs. True Parameters: first %d of q=%d edges per state. Each state has q intercepts (be
      n_show, dat$q),
    col.names = c("State", "Edge", "True beta0", "Est. beta0", "True beta1", "Est. beta1")
  ) %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE) %>%
  collapse_rows(columns = 1, valign = "top")

```

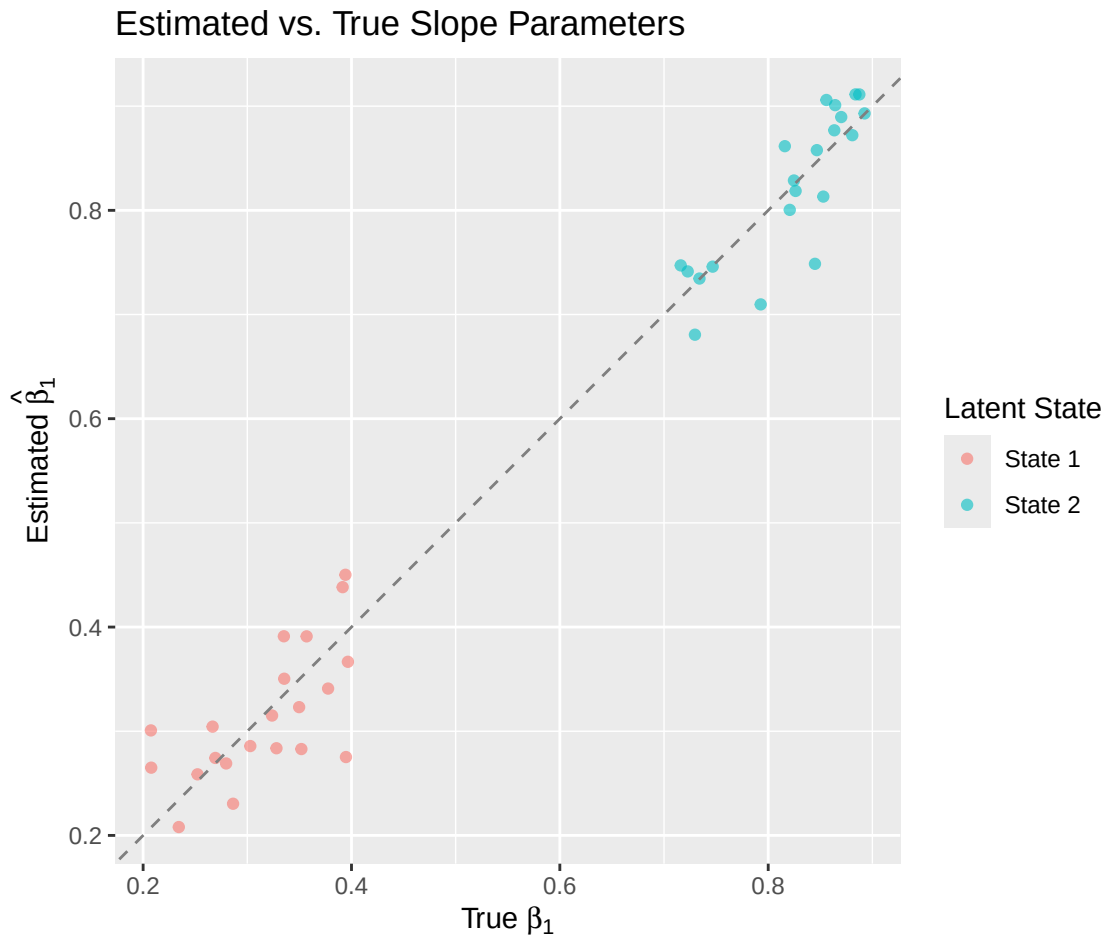
Table 1: Estimated vs. True Parameters: first 10 of $q=21$ edges per state. Each state has q intercepts (β_0) and q slopes (β_1), one per edge.

State	Edge	True beta0	Est. beta0	True beta1	Est. beta1
State 1	1	0.6855	0.6934	0.2075	0.3008
State 1	2	-0.2823	-0.2038	0.3947	0.2752
State 1	3	0.1816	0.1312	0.2864	0.2304
State 1	4	0.3164	0.3579	0.3915	0.4383
State 1	5	0.2021	0.1339	0.3776	0.3410
State 1	6	-0.0531	-0.0952	0.3280	0.2837
State 1	7	0.7558	0.7876	0.3942	0.4502
State 1	8	-0.0473	-0.0228	0.3238	0.3151
State 1	9	1.0092	1.0210	0.2667	0.3044
State 1	10	-0.0314	-0.0611	0.2693	0.2744
State 2	1	1.8791	1.8859	0.8838	0.9113
State 2	2	1.1366	1.0647	0.8925	0.8930
State 2	3	0.8159	0.7939	0.7467	0.7459

State 2	4	1.7164	1.7204	0.8449	0.7487
State 2	5	1.0943	1.0710	0.8807	0.8721
State 2	6	2.2221	2.1625	0.8207	0.8004
State 2	7	1.2843	1.2952	0.8263	0.8186
State 2	8	1.8278	1.8108	0.8875	0.9112
State 2	9	1.6610	1.7289	0.8701	0.8895
State 2	10	1.1081	1.1166	0.8160	0.8616

```
# scatter plot of all estimated vs true beta1 values
plot_df = tibble(
  true = c(dat$true_params$beta1[1,], dat$true_params$beta1[2,]),
  estimated = c(aligned$beta1[1,], aligned$beta1[2,]),
  state = rep(c("State 1", "State 2"), each = dat$q)
)

ggplot(plot_df, aes(x = true, y = estimated, color = state)) +
  geom_point(alpha = 0.6) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey50") +
  labs(
    title = "Estimated vs. True Slope Parameters",
    x = expression("True " * beta[1]),
    y = expression("Estimated " * hat(beta)[1]),
    color = "Latent State"
  ) +
  coord_equal()
```



```
cat(sprintf("True mixing proportions: %s\n",
            paste(round(dat$true_params$pi, 3), collapse = ", ")))
```

```
## True mixing proportions: 0.5, 0.5
```

```
cat(sprintf("Estimated mixing proportions: %s\n",
            paste(round(aligned$pi, 3), collapse = ", ")))
```

```
## Estimated mixing proportions: 0.44, 0.56
```

```
# classification accuracy
gamma = em_estep(dat$s_mat, dat$r_mat, fit$params)
z_hat = apply(gamma, 1, which.max)

cat(sprintf("Estimated state frequencies: %s\n", paste(table(z_hat), collapse = ", ")))
```

```
## Estimated state frequencies: 44, 56
```

Model Selection

```

sel = select_K(dat$s_mat, dat$r_mat, K_max = 4, n_init = 10)

sel$table %>%
  mutate(across(c(loglik, AIC, BIC), ~round(.x, 2))) %>%
  kable(caption = "Model Selection: AIC and BIC across K",
        col.names = c("K", "Log-lik", "# Params", "AIC", "BIC", "Converged", "Iterations")) %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)

```

Table 2: Model Selection: AIC and BIC across K

K	Log-lik	# Params	AIC	BIC	Converged	Iterations
1	-2425.58	63	4977.16	5141.28	TRUE	20
2	-482.13	127	1218.26	1549.11	TRUE	20
3	-422.55	191	1227.10	1724.69	TRUE	23
4	-374.50	255	1258.99	1923.31	TRUE	20

```

cat(sprintf("BIC-selected K: %d (true K = 2)\n",
  sel$table$K[which.min(sel$table$BIC)]))

```

BIC-selected K: 2 (true K = 2)

Full Simulation Study (Cluster)

The simulation is run via:

```

cd "final project"
sbatch simulations/run_sim.sh

```

Results

Bootstrap Inference

```

set.seed(2026)
B_boot = 200

boot_params = vector("list", B_boot)
n_boot = nrow(dat$s_mat)

for (b in 1:B_boot) {
  idx = sample(n_boot, n_boot, replace = TRUE)
  s_b = dat$s_mat[idx, , drop = FALSE]
  r_b = dat$r_mat[idx, , drop = FALSE]

  fit_b = em_lcr(s_b, r_b, K = 2, n_init = 1,
    init_params = fit$params, tol = 1e-5, max_iter = 300)

  if (fit_b$converged || fit_b$n_iter > 10) {
    aligned_b = align_labels(fit_b$params, fit$params, K = 2)
  }
}

```

```

    boot_params[[b]] = aligned_b
  }
}

# drop any NULL (non-converged)
boot_params = Filter(Negate(is.null), boot_params)
B_actual = length(boot_params)
cat(sprintf("Successful bootstrap replicates: %d / %d\n", B_actual, B_boot))

```

```
## Successful bootstrap replicates: 200 / 200
```

```

# mixing proportion inference
pi_full = fit$params$pi
pi_boot_mat = do.call(rbind, lapply(boot_params, `[`, "pi"))

pi_ci = apply(pi_boot_mat, 2, quantile, probs = c(0.025, 0.975))
pi_se = apply(pi_boot_mat, 2, sd)

pi_summary_demo = data.frame(
  State      = paste("State", 1:2),
  Estimate   = round(pi_full, 4),
  SE         = round(pi_se, 4),
  CI_Lower   = round(pi_ci[1, ], 4),
  CI_Upper   = round(pi_ci[2, ], 4)
)

pi_summary_demo %>%
  kable(caption = sprintf("Bootstrap Inference: Mixing Proportions (%d replicates, n=100, p=6, K=2)", B_boot),
        col.names = c("State", "Estimate", "Bootstrap SE", "95% CI Lower", "95% CI Upper")) %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE)

```

Table 3: Bootstrap Inference: Mixing Proportions (200 replicates, n=100, p=6, K=2)

State	Estimate	Bootstrap SE	95% CI Lower	95% CI Upper
State 1	0.44	0.0482	0.3595	0.5400
State 2	0.56	0.0482	0.4600	0.6405

```

# beta1 inference for first 8 edges per state
beta1_full = fit$params$beta1
beta1_boot_arr = array(
  do.call(rbind, lapply(boot_params, function(x) c(x$beta1))),
  dim = c(B_actual, 2, dat$q)
)

n_show_b = 8
beta1_rows = list()
for (k in 1:2) {
  for (j in 1:n_show_b) {
    bv = beta1_boot_arr[, k, j]
    beta1_rows[[length(beta1_rows) + 1]] = data.frame(

```

```

    State    = paste("State", k),
    Edge      = j,
    Estimate  = round(beta1_full[k, j], 4),
    SE        = round(sd(bv), 4),
    CI_Lower  = round(quantile(bv, 0.025), 4),
    CI_Upper  = round(quantile(bv, 0.975), 4)
  )
}
}
beta1_table = do.call(rbind, beta1_rows)

beta1_table %>%
  kable(caption = "Bootstrap Inference: Slope Parameters beta1 (first 8 edges per state)",
        col.names = c("State", "Edge", "Estimate", "Bootstrap SE", "95% CI Lower", "95% CI Upper")) %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE) %>%
  collapse_rows(columns = 1, valign = "top")

```

Table 4: Bootstrap Inference: Slope Parameters beta1 (first 8 edges per state)

	State	Edge	Estimate	Bootstrap SE	95% CI Lower	95% CI Upper
2.5%	State 1	1	0.3008	0.0604	0.1766	0.4065
2.5%1	State 1	2	0.2752	0.0698	0.1430	0.3996
2.5%2	State 1	3	0.2304	0.0437	0.1510	0.3196
2.5%3	State 1	4	0.4383	0.0306	0.3840	0.5037
2.5%4	State 1	5	0.3410	0.0478	0.2699	0.4436
2.5%5	State 1	6	0.2837	0.0629	0.1759	0.4058
2.5%6	State 1	7	0.4502	0.0355	0.3749	0.5063
2.5%7	State 1	8	0.3151	0.0358	0.2449	0.3815
2.5%8	State 2	1	0.9113	0.0646	0.7969	1.0492
2.5%9	State 2	2	0.8930	0.0358	0.8293	0.9672
2.5%10	State 2	3	0.7459	0.0412	0.6650	0.8243
2.5%11	State 2	4	0.7487	0.0421	0.6735	0.8298
2.5%12	State 2	5	0.8721	0.0353	0.8039	0.9378
2.5%13	State 2	6	0.8004	0.0632	0.6758	0.9408
2.5%14	State 2	7	0.8186	0.0462	0.7288	0.9105
2.5%15	State 2	8	0.9112	0.0545	0.7955	1.0102

```

# Bootstrap distribution of pi_1 and beta1[1,1]
pi1_boot = pi_boot_mat[, 1]
b1_edge1_boot = beta1_boot_arr[, 1, 1]

plot_df_boot = bind_rows(
  tibble(param = "pi_1 (mixing proportion, State 1)", value = pi1_boot,
    estimate = pi_full[1]),

```



```

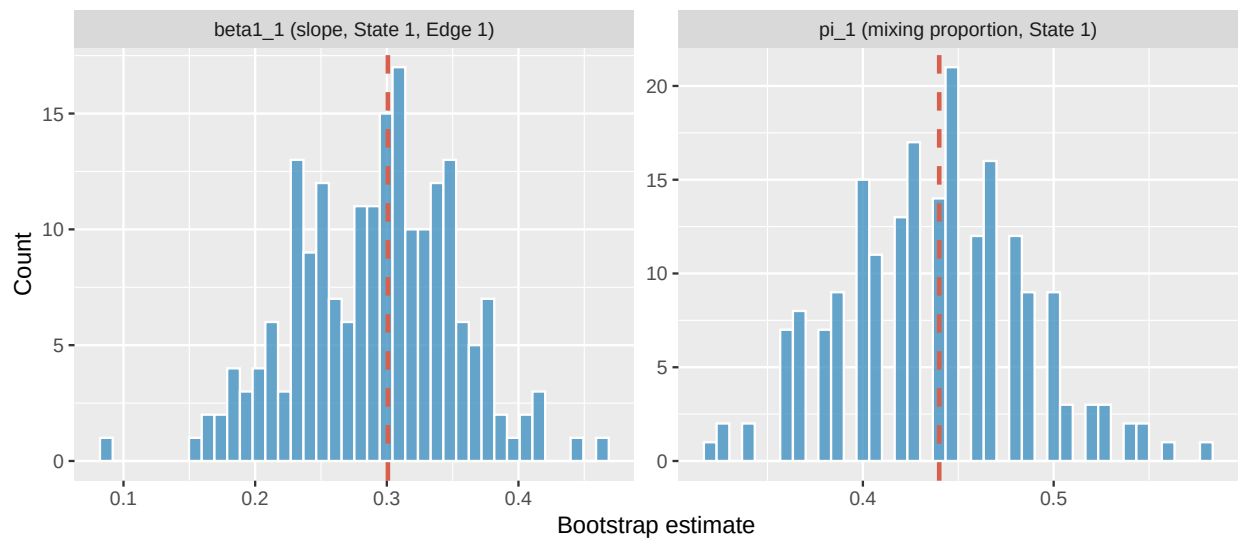
  tibble(param = "beta1_1 (slope, State 1, Edge 1)", value = b1_edge1_boot,
         estimate = beta1_full[1, 1])
)

ggplot(plot_df_boot, aes(x = value)) +
  geom_histogram(bins = 40, fill = "#4393C3", color = "white", alpha = 0.8) +
  geom_vline(aes(xintercept = estimate), color = "#D6604D", linewidth = 1, linetype = "dashed") +
  facet_wrap(~param, scales = "free") +
  labs(
    title = sprintf("Bootstrap Distributions (%d replicates)", B_actual),
    subtitle = "Red dashed line = full-data estimate",
    x = "Bootstrap estimate", y = "Count"
  )
)

```

Bootstrap Distributions (200 replicates)

Red dashed line = full-data estimate



Simulation Study Results

```

sim_files = list.files(here::here("results", "20260414"), pattern = "\\..RDA$",
                      full.names = TRUE)

all_results = list()
for (f in sim_files) {
  load(f)
  all_results = c(all_results, results)
}

sim_df = bind_rows(all_results) %>%
  mutate(
    n = factor(n, levels = c(50, 100, 200)),
    p_label = paste0("p = ", p, " (q = ", p*(p+1)/2, ")"),
    K_label = paste0("K = ", K_true)
  )
)

```

```

sim_df %>%
  group_by(n, p, K_true) %>%
  summarise(
    bic_accuracy = mean(bic_correct),
    mean_beta1_err = mean(beta1_frob_err),
    mean_ari = mean(ari),
    pct_converged = mean(converged) * 100,
    .groups = "drop"
  ) %>%
  arrange(K_true, p, n) %>%
  mutate(across(where(is.numeric), ~round(.x, 3))) %>%
  kable(
    caption = "Simulation Study Summary (500 replicates per scenario)",
    col.names = c("n", "p", "True K", "BIC Accuracy",
                  "Mean ||beta1 error||_F", "Mean ARI", "% Converged")
  ) %>%
  kable_styling(latex_options = c("striped", "hold_position"), full_width = FALSE) %>%
  pack_rows("K = 2", 1, 6) %>%
  pack_rows("K = 3", 7, 12) %>%
  pack_rows("K = 4", 13, 18)

```

Table 5: Simulation Study Summary (500 replicates per scenario)

n	p	True K	BIC Accuracy	Mean beta1 error _F	Mean ARI	% Converged
K = 2						
50	6	2	0.932	0.442	1	100
100	6	2	1.000	0.299	1	100
200	6	2	1.000	0.206	1	100
50	15	2	0.926	1.059	1	100
100	15	2	1.000	0.716	1	100
200	15	2	1.000	0.497	1	100
K = 3						
50	6	3	0.598	0.704	1	100
100	6	3	0.992	0.456	1	100
200	6	3	1.000	0.313	1	100
50	15	3	0.526	1.679	1	100
100	15	3	0.994	1.110	1	100
200	15	3	1.000	0.757	1	100
K = 4						
50	6	4	0.282	1.055	1	100
100	6	4	0.924	0.627	1	100
200	6	4	0.998	0.424	1	100
50	15	4	0.202	2.917	1	100
100	15	4	0.976	1.506	1	100
200	15	4	0.996	1.022	1	100

```

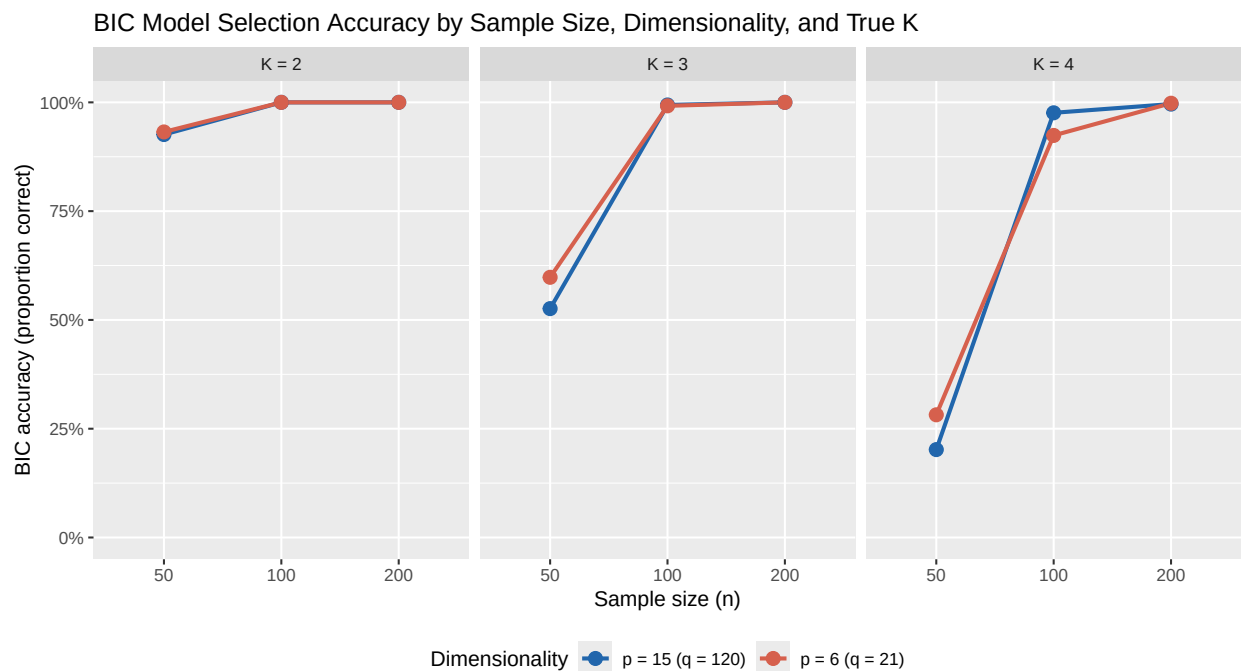
sim_df %>%
  group_by(n, p_label, K_label) %>%
  summarise(bic_accuracy = mean(bic_correct), .groups = "drop") %>%
  ggplot(aes(x = n, y = bic_accuracy, color = p_label, group = p_label)) +
  geom_line(linewidth = 1) +

```

```

geom_point(size = 3) +
facet_wrap(~K_label) +
scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1)) +
scale_color_manual(values = c("#2166AC", "#D6604D")) +
labs(
  title = "BIC Model Selection Accuracy by Sample Size, Dimensionality, and True K",
  x = "Sample size (n)", y = "BIC accuracy (proportion correct)",
  color = "Dimensionality"
) +
theme(legend.position = "bottom")

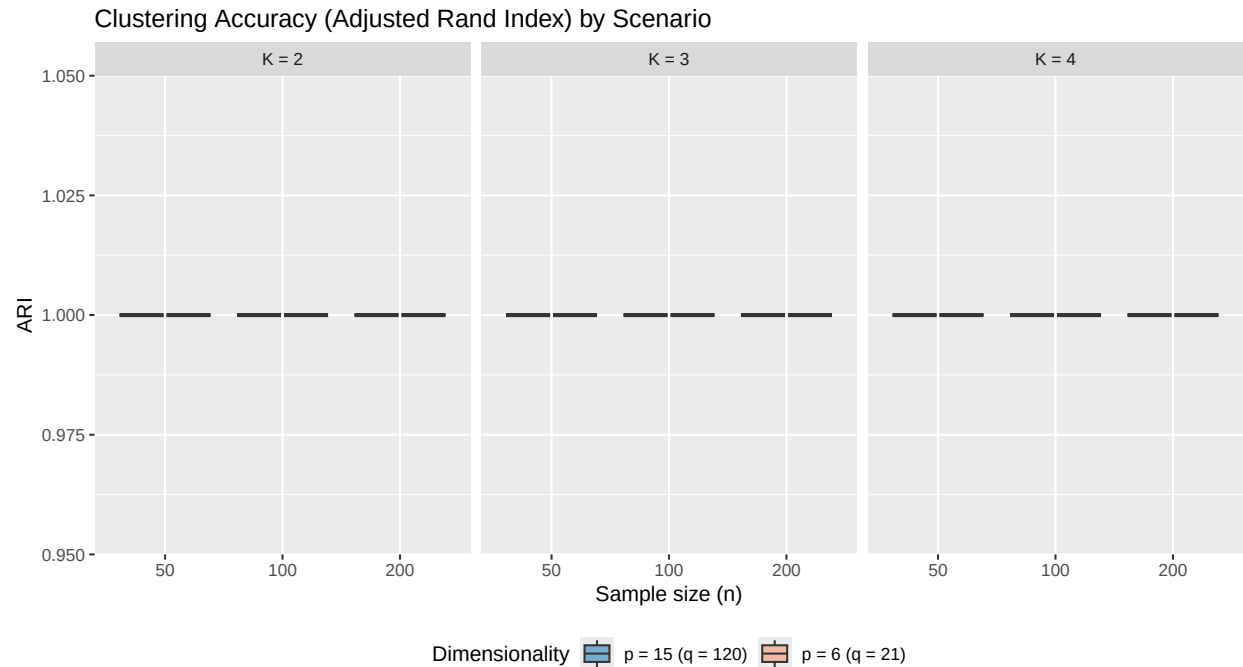
```



```

sim_df %>%
  ggplot(aes(x = n, y = ari, fill = p_label)) +
  geom_boxplot(outlier.size = 0.5, alpha = 0.7, position = position_dodge(width = 0.8)) +
  facet_wrap(~K_label) +
  scale_fill_manual(values = c("#4393C3", "#F4A582")) +
  labs(
    title = "Clustering Accuracy (Adjusted Rand Index) by Scenario",
    x = "Sample size (n)", y = "ARI",
    fill = "Dimensionality"
  ) +
  theme(legend.position = "bottom")

```



```
sim_df %>%
  group_by(n, p_label, K_label) %>%
  summarise(mean_err = mean(beta1_frob_err),
            se_err   = sd(beta1_frob_err) / sqrt(n()),
            .groups  = "drop") %>%
  ggplot(aes(x = n, y = mean_err, color = p_label, group = p_label)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_err - 1.96 * se_err,
                    ymax = mean_err + 1.96 * se_err), width = 0.1) +
  facet_wrap(~K_label, scales = "free_y") +
  scale_color_manual(values = c("#2166AC", "#D6604D")) +
  labs(
    title = "Parameter Estimation Error: Mean Frobenius Norm of beta1",
    subtitle = "Error bars = +/- 1.96 x Monte Carlo SE",
    x = "Sample size (n)", y = "Mean Frobenius error",
    color = "Dimensionality"
  ) +
  theme(legend.position = "bottom")
```

Parameter Estimation Error: Mean Frobenius Norm of beta1

Error bars = $\pm 1.96 \times$ Monte Carlo SE

